



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر
پایانترم فناوری رباتیک

عنوان:

پیاده‌سازی الگوریتم‌های هوش مصنوعی در رباتیک

نگارش

محمدرضا منعمیان ۴۰۲۱۰۶۶۰۴

بهمن ماه ۱۴۰۴

فهرست مطالب

۲	۱	قسمت اول : پیاده سازی localization
۲	۱-۱	پیکربندی سنسور لایدار در URDF
۳	۲-۱	مدیریت تبدیل مختصاتی : TF Static
۳	۳-۱	اجرا و تحلیل نتایج
۴	۲	قسمت دوم : پیاده سازی A*
۴	۱-۲	توضیحی درباره الگوریتم
۴	۲-۲	منطق عملکرد سیستم و تعاملات ROS ۲
۵	۳-۲	ایمنی حرکت و در نظر گرفتن ابعاد ربات
۵	۴-۲	پیاده سازی کد اصلی
۶	۵-۲	نتایج و تحلیل تصویری
۷	۳	قسمت سوم : پیاده سازی سرویس کنترل به صورت کلاسیک
۷	۱-۳	تئوری کنترل کننده PID و منطق Lookahead
۷	۲-۳	تحلیل فنی پیاده سازی و ارتباطات ROS ۲
۸	۳-۳	جلوگیری از برخورد و منطق اتمام مسیر
۸	۴-۳	دستورالعمل اجرا (Deployment) Procedure
۹	۴	معماری یادگیری تقویتی : Action State و Reward
۹	۱-۴	تعریف فضاها (Spaces)
۹	۲-۴	طراحی تابع پاداش (Reward) Function
۱۰	۳-۴	ساختار نرم افزاری و فایل های پروژه
۱۰	۴-۴	نتیجه گیری و مستندات ویدئویی

۱ قسمت اول : پیاده سازی localization

در این پروژه، از سیستم nav2_map_server برای بارگذاری محیط استفاده شده است. نقشه در پوشه maps و در دو قالب زیر قرار داده شده است:

- **depot.pgm**: فایل تصویری شامل موانع و فضاهای آزاد محیط.
- **depot.yaml**: فایل تنظیمات متنی که اطلاعات مقیاس (Resolution) و مبدأ مختصات (Origin) را تعریف می‌کند.

۱-۱ پیکربندی سنسور لایدار در URDF

برای شبیه‌سازی دقیق سنسور در محیط Gazebo، تگ مربوط به سنسور rplidar_c1 در فایل URDF به صورت زیر اصلاح گردید. این پیکربندی شامل تنظیماتی برای نرخ به‌روزرسانی (۲۰ HZ) تعداد نمونه‌ها (۷۲۰ نقطه) و بازه اندازه‌گیری (۱۲ متر) می‌باشد:

```
1 <gazebo reference="rplidar_c1">
2   <sensor type="gpu_ray" name="rplidar_c1_sensor">
3     <always_on>1</always_on>
4     <visualize>true</visualize>
5     <update_rate>20</update_rate>
6     <topic>scan</topic>
7     <frame_id>rplidar_c1</frame_id>
8     <lidar>
9       <scan>
10        <horizontal>
11          <samples>720</samples>
12          <min_angle>-3.14</min_angle>
13          <max_angle>3.14</max_angle>
14        </horizontal>
15      </scan>
16      <range>
17        <min>0.15</min>
18        <max>12.0</max>
19      </range>
20    </lidar>
21  </sensor>
22 </gazebo>
```

۲-۱ مدیریت تبدیل مختصاتی : TF Static

به دلیل تفاوت نام فریم‌های تعریف شده در سنسور Gazebo و سیستم مختصات ربات، از دستور زیر در یک ترمینال مجزا استفاده شد:

```
1 ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 rplidar_c1  
robot/base_link/rplidar_c1_sensor
```

این دستور یک تبدیل ثابت با جابجایی و دوران صفر ایجاد می‌کند تا زنجیره TF بین فریم فیزیکی لایدار و نام سنسور در Gazebo برقرار شود و الگوریتم AMCL بتواند داده‌های لیزر را به درستی تفسیر کند.

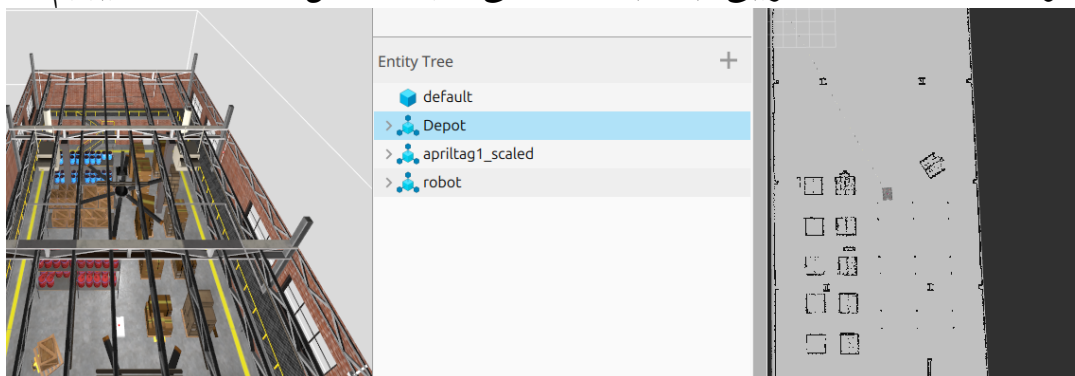
۳-۱ اجرا و تحلیل نتایج

فرآیند محلی‌سازی با طی مراحل زیر در محیط RViz با موفقیت انجام شد:

۱. مقداردهی اولیه: با استفاده از ابزار Estimate Pose 2D، تخمین اولیه موقعیت ربات به الگوریتم داده شد.

۲. همگرایی الگوریتم: الگوریتم AMCL با استفاده از فیلتر ذرات، موقعیت دقیق ربات را بر اساس داده‌های تاپیک scan/ استخراج کرد.

۳. خروجی سیستم: مختصات دقیق ربات به صورت مداوم روی تاپیک amcl_pose منتشر شده البته هر کاری کردیم که ذرات prarticle را بتوانیم با استفاده از خواندن از تاپیک par-ticlecloud در محیط rviz نشان دهیم نتوانستیم. در زیر عکسی از rviz را که در آن مپ لود شده است و مان تقریبی ربات با مقدار دهی اولیه مشخص شده است را میبینیم.



شکل ۱: نمودار به دست آمده از تغییر

۲ قسمت دوم: پیاده سازی A^*

۱-۲ توضیحی درباره الگوریتم

الگوریتم A^* یکی از پیشرفته‌ترین روش‌های جستجوی مسیر است که به دلیل ترکیب دقت الگوریتم Dijkstra و سرعت روش‌های Heuristic شناخته می‌شود. این الگوریتم با استفاده از تابع هزینه زیر، بهینه‌ترین مسیر را تضمین می‌کند:

$$f(n) = g(n) + h(n) \quad (۱)$$

که در آن $g(n)$ هزینه واقعی از مبدأ تا گره جاری و $h(n)$ تخمین فاصله تا هدف است. در این سوال از فاصله اقلیدسی به عنوان تابع Heuristic استفاده شده است تا Admissibility الگوریتم (عدم تخمین بیش از حد هزینه) و در نتیجه یافتن کوتاه‌ترین مسیر ممکن تضمین شود.

۲-۲ منطق عملکرد سیستم و تعاملات ROS

واحد planner به گونه‌ای طراحی شده است که به عنوان یک هماهنگ‌کننده مرکزی عمل کند. فرآیند تولید مسیر به شرح زیر است:

- دریافت نقشه Map Grid: داده‌ها از تاپیک /map دریافت می‌شوند. پیکسل‌های با مقدار ۱۰۰ به عنوان موانع صلب و مقادیر دیگر به عنوان فضای آزاد تحلیل می‌شوند. همچنین اگر پیکسلی ۱- باشد به معنی ناشناخته بودن آن است. توجه داریم که این تاپیک پیکسل‌ها را به صورت لیست یک بعدی می‌دهد برای همین برای استفاده از آن باید این لیست یک بعدی را به صورت ماتریس نگاه کرد. با کامند زیر می‌توانیم نقشه را تحلیل کنیم:

```
> ros2 topic echo /map --once --field data | grep -oE "0|100|-1" | sort | uniq -c
179481 0
5947 100
```

شکل ۲: تحلیل نقشه

- تعیین مبدأ: موقعیت لحظه‌ای ربات به صورت خودکار از طریق سابسکریپشن به تاپیک /amcl_pose استخراج می‌گردد.
- درخواست مقصد Call Service: برای بهینه‌سازی مصرف پردازنده، مسیریابی تنها با فراخوانی سرویس /plan_path آغاز می‌شود. نمونه دستور فراخوانی:

```

1 ros2 service call /plan_path nav_msgs/srv/GetPlan "{goal: {pose: {position: {x: 22.3, y:
    7.13}}}}}"

```

۳-۲ ایمنی حرکت و در نظر گرفتن ابعاد ربات

یکی از نوآوری‌های این بخش، پیاده‌سازی تابع `is_valid` با منطق **Layer Inflation** مجازی است. با توجه به اینکه عبور مسیر از مجاورت لبه دیوارها منجر به برخورد فیزیکی می‌شود، شعاع ربات (۳۰ سانتی‌متر) در محاسبات لحاظ شده است. بر اساس اینکه هر خانه در نقشه چقدر اندازه دارد بر حسب شعاع ربات خانه‌های اطراف نیز بررسی می‌شود تا فقط مرکز ربات در نظر گرفته نشود و ابعاد ربات در اینجا لحاظ شود.

```

1 def is_valid(self, node):
2     # ... check map boundaries ...
3     # Calculate radius in pixels based on map resolution
4     radius_cells = int(math.ceil(self.robot_radius / self.map_info.resolution))
5
6     # Square check around the node to ensure clearance
7     for i in range(-radius_cells, radius_cells + 1):
8         for j in range(-radius_cells, radius_cells + 1):
9             if (i**2 + j**2) <= (radius_cells**2):
10                 idx = (ny + j) * width + (nx + i)
11                 if self.map_data[idx] > 50: # Wall detected
12                     return False
13     return True

```

۴-۲ پیاده‌سازی کد اصلی

بخش مرکزی الگوریتم جستجو در زبان پایتون به شرح زیر است.

```

1 def a_star(self, start, goal):
2     open_set = []
3     heapq.heappush(open_set, (0, start))
4     came_from = {}
5     g_score = {start: 0}
6
7     while open_set:
8         _, current = heapq.heappop(open_set)
9
10

```

```

11     if current == goal:
12         return self.reconstruct_path(came_from, current)
13     # ... logic for neighbors exploration ...

```

۵-۲ نتایج و تحلیل تصویری

برای اینکه از این سرویس استفاده کنیم باید در یک ترمینال launchfile مربوط به gazebo را اجرا کنیم و در ترمینال دیگر لانچ فایل مربوط به localization ، planner ، در مرحله آخر به صورت گفته شده نقطه شروع را از rviz و نقطه هدف را به صورت کامند call service به کد میدهیم.

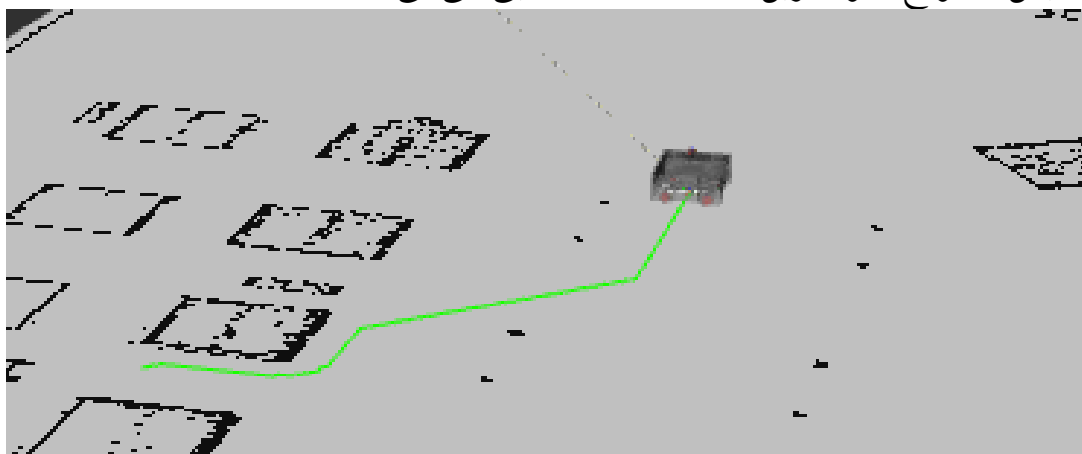
```

~/robot1/vr/sr/robotic_course main 15 718 ros2 service call /plan_path n
v_msgs/srv/GetPlan (start: {}, goal: {header: {frame_id: 'map'}, pose: {positi
n: {x: 22.3, y: 5.14, z: 0.0}}, tolerance: 0.0})

```

شکل ۳: اجرای کامند های مربوطه

پس از اجرای الگوریتم، مسیر تولید شده در تایپیک /planned_path منتشر گردیده و در نرم افزار RViz نمایش داده شد. همان طور که در نتایج مشخص است، مسیر کاملاً پیوسته بوده و با حفظ فاصله ایمن از موانع، کوتاه ترین راه را تا مقصد نهایی طی می کند.



شکل ۴: نتایج الگوریتم a*

۳ قسمت سوم: پیاده سازی سرویس کنترل به صورت کلاسیک

۱-۳ تئوری کنترل کننده PID و منطق Lookahead

کنترل کننده PID یک مکانیزم کنترلی بر پایه بازخورد (Feedback) است که خطای بین مقدار مطلوب و مقدار فعلی را اصلاح می کند. در این پروژه از نسخه PD (Proportional-Derivative) استفاده شده است:

- **بهره تناسبی (K_p):** قدرت واکنش ربات به خطای زاویه ای فعلی را تعیین می کند.
- **بهره مشتق گیر (K_d):** با بررسی نرخ تغییرات خطا، از نوسانات شدید (Overshoot) جلوگیری کرده و حرکت را نرم می کند.

برای تعقیب نرم مسیر، از استراتژی **Lookahead** استفاده شده است؛ به این معنا که ربات به جای تلاش برای انطباق بر نزدیک ترین نقطه، هدفی را در چند گام جلوتر (در اینجا ۵ گام) انتخاب کرده و به سمت آن جهت گیری می کند.

۲-۳ تحلیل فنی پیاده سازی و ارتباطات ROS ۲

گره `pid_line_follower` به عنوان پل ارتباطی بین نقشه، مکان یاب و موتورهای ربات عمل می کند.

- **ورودی ها (Subscriptions):**

— `/planned_path`: دریافت لیست نقاط مسیر تولید شده توسط A^* .

— `/amcl_pose`: دریافت موقعیت و زاویه لحظه ای ربات از سیستم مکان یاب.

- **خروجی (Publisher):**

— `/cmd_vel`: ارسال دستورات سرعت خطی (v) و سرعت زاویه ای (ω) به درایور ربات.

در بدنه تابع `odom_callback`، ابتدا زاویه کواترنيون ربات به زاویه اوایلر (Yaw) تبدیل شده و سپس خطای زاویه ای نسبت به نقطه هدف محاسبه می گردد:

```
1 # PD Control Calculation
2 desired_yaw = math.atan2(target_y - y, target_x - x)
3 error = desired_yaw - yaw
```



```

4
5 # Normalize error to [-pi, pi]
6 while error > math.pi: error -= 2 * math.pi
7 while error < -math.pi: error += 2 * math.pi
8
9 angular_z = (self.kp * error) + (self.kd * (error - self.prev_error))

```

۳-۳ جلوگیری از برخورد و منطق اتمام مسیر

ربات با سرعت ثابت 2.0 m/s حرکت می‌کند. در هر چرخه، فاصله تا نقطه پایانی مسیر محاسبه می‌شود. طبق کد، اگر فاصله ربات تا انتهای مسیر کمتر از ۲۰ سانتی‌متر باشد، سرعت خطی و زاویه‌ای صفر شده و وضعیت REACHED TARGET گزارش می‌شود.

۴-۳ دستورالعمل اجرا (Deployment Procedure)

برای اجرای موفقیت‌آمیز سیستم ناوبری، مراحل زیر به ترتیب طی شدند:

۱. تولید مسیر جهانی: ابتدا با استفاده از گره a_star_planner و فراخوانی سرویس مربوطه، یک مسیر بهینه از مبدأ تا مقصد ایجاد و روی تاپیک /planned_path منتشر گردید.

۲. فعال‌سازی کنترلر: سپس با اجرای دستور زیر، کنترلر PD فعال شد:

```

1 ros2 run robot_description pid_line_follower

```

۳. تعقیب مسیر: کنترلر با دریافت نقاط، نزدیک‌ترین گره را پیدا کرده و با انتخاب ۵ گام جلوتر به عنوان هدف لحظه‌ای، ربات را با دقت بالا روی مسیر هدایت کرد.

```

ros2 launch robot_description pid_line_follower.launch.py
[INFO] [launch]: All log files can be found below /home/mohammadreza/.ros/log/2026-02-01-20-03-13-884410-mohammadreza-Vlvobook-ASUSLaptop-X1502ZA-R1502ZA-2092519
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [pid_follower-1]: process started with pid [2092522]
pid_follower-1 [INFO] [1769963595.120924580] [pid_line_follower_node]: --- STARTING PID NODE ---
pid_follower-1 [INFO] [1769963595.121685202] [pid_line_follower_node]: 1. Waiting for Path from /planned_path...
pid_follower-1 [INFO] [1769963595.122261129] [pid_line_follower_node]: 2. Waiting for Pose from /amcl_pose...
pid_follower-1 [INFO] [1769963620.020272570] [pid_line_follower_node]: ✓ PATH RECEIVED! Points: 195
pid_follower-1 [INFO] [1769963620.029014911] [pid_line_follower_node]: Start Point: (16.0250000238791108, 8.5750000127777457)
pid_follower-1 [INFO] [1769963620.02913013] [pid_line_follower_node]: End Point: (22.2750000331923366, 5.1250000076368451)
pid_follower-1 [INFO] [1769963626.860354999] [pid_line_follower_node]: ⚡ Driving: Err=0.26 | Cmd=[v=0.2, w=0.39]
pid_follower-1 [INFO] [1769963628.261555053] [pid_line_follower_node]: ⚡ Driving: Err=0.17 | Cmd=[v=0.2, w=0.13]
pid_follower-1 [INFO] [1769963631.170818322] [pid_line_follower_node]: ⚡ Driving: Err=0.16 | Cmd=[v=0.2, w=0.16]
pid_follower-1 [INFO] [1769963634.133820366] [pid_line_follower_node]: ⚡ Driving: Err=0.13 | Cmd=[v=0.2, w=0.11]
pid_follower-1 [INFO] [1769963637.868542240] [pid_line_follower_node]: ⚡ Driving: Err=0.29 | Cmd=[v=0.2, w=-0.49]
pid_follower-1 [INFO] [1769963639.356063105] [pid_line_follower_node]: ⚡ Driving: Err=0.11 | Cmd=[v=0.2, w=-0.03]
pid_follower-1 [INFO] [1769963642.331396205] [pid_line_follower_node]: ⚡ Driving: Err=0.25 | Cmd=[v=0.2, w=-0.32]
pid_follower-1 [INFO] [1769963643.78480918] [pid_line_follower_node]: ⚡ Driving: Err=0.03 | Cmd=[v=0.2, w=0.16]
pid_follower-1 [INFO] [1769963646.618842452] [pid_line_follower_node]: ⚡ Driving: Err=0.37 | Cmd=[v=0.2, w=-0.57]
pid_follower-1 [INFO] [1769963648.037105782] [pid_line_follower_node]: ⚡ Driving: Err=0.32 | Cmd=[v=0.2, w=-0.30]
pid_follower-1 [INFO] [1769963649.175988521] [pid_line_follower_node]: ⚡ Driving: Err=0.17 | Cmd=[v=0.2, w=-0.09]

```

شکل ۵: اجرای الگوریتم pid

۴ معماری یادگیری تقویتی: Reward و Action State

در این تمرین از الگوریتم $\text{Deep Gradient Policy Deterministic (DDPG)}$ برای آموزش ربات جهت تعقیب مسیر استفاده شده است. انتخاب این الگوریتم به دلیل توانایی آن در کار با فضاهاى اکشن پیوسته (Continuous) است.

۱-۴ تعریف فضاها (Spaces)

- فضای وضعیت (State): شامل یک بردار ۴ بُعدی $[LatErr, HeadErr, v, \omega]$ است.
 - $LatErr$ و $HeadErr$: خطای جانبی و زاویه‌ای نسبت به مسیر A^* که به ربات می‌فهماند چقدر از مسیر منحرف شده است.
 - v و ω : سرعت‌های فعلی ربات برای درک اینرسی و پویایی حرکت.
- فضای اکشن (Action): شامل دو خروجی پیوسته در بازه $[-1, 1]$ است که پس از نرمال‌سازی به سرعت خطی (۰ تا ۰/۴ متر بر ثانیه) و سرعت زاویه‌ای تبدیل می‌شوند.

۲-۴ طراحی تابع پاداش (Reward Function)

تابع پاداش به گونه‌ای طراحی شده تا ربات تعادلی بین "سرعت" و "دقت" برقرار کند:

$$R = (v \times 2) - (2 \times |LatErr|) - (1 \times |HeadErr|) \quad (2)$$

- تشویق سرعت: ترم اول ربات را ترغیب می‌کند تا مسیر را سریع‌تر طی کند.
- جریمه انحراف: ترم‌های دوم و سوم ربات را تنبیه می‌کنند تا دقیقاً روی خط باقی بماند.
- پاداش نهایی: در صورت رسیدن به مقصد (Goal) پاداش بزرگ $+100$ و در صورت تصادف (انحراف بیش از ۸۰ متر)، جریمه -20 اعمال می‌گردد. و همچنین ربات ریست می‌شود ریست شدن آن نیز به این صورت است که به ابتدای مسیر ترنسفر می‌شود توجه داریم که بعد از ۱۰۰۰ گام نیز خود به خود اپیزود را تمام می‌کنیم.

۳-۴ ساختار نرم‌افزاری و فایل‌های پروژه

پیاده‌سازی این بخش شامل ۴ فایل کلیدی است که وظایف تفکیک شده‌ای دارند:

۱. `gym_gazebo_env.py`: زیرساخت اصلی برای ارتباط با Gazebo. وظیفه ریست کردن محیط، اسپاون ربات و همگام‌سازی زمان شبیه‌سازی با کد پایتون را دارد.

۲. `line_following_env.py`: تعریف اختصاصی محیط یادگیری. در این فایل منطق محاسبه خطاها (بر اساس مسیر A^* و تابع پاداش پیاده شده است).

۳. `ddpg_agent.py`: پیاده‌سازی مغز متفکر ربات شامل شبکه‌های عصبی Actor و Critic، حافظه تجربه (Replay Buffer) و الگوریتم به‌روزرسانی وزن‌ها.

۴. `train_robot.py`: اسکریپت اصلی برای شروع حلقه آموزش، مدیریت اپیزودها و ذخیره‌سازی مدل‌های نهایی (.pth).

۴-۴ نتیجه‌گیری و مستندات ویدئویی

نتیجه‌ای که از ترین گرفتیم خیلی مناسب نبود بعد از ۱۰۰ اپیزود نیز همچنان ربات نمیتوانست به صورت مناسب و درست یک مسیر صاف را برود به این خاطر که نیاز بیشتر به اپیزودهای بیشتر داشت که زمان اجازه نداد. همچنین الگوریتمی که نوشتیم برای یک مسیر است و هر بار که مسیر عوض شود باید دوباره روی آن مسیر ترین شود کمی استفاده از RL در اینجا چالشی بود.